

Formato de Entrega de Tareas y Exámenes

Índice

1. Foros de Discusión	2
2. Foros de Discusión por Pares	2
3. Forma de Entrega del Código	2
4. Entrega en la Plataforma	3
5. Configuración de <code>.gitignore</code>	3
6. Exámenes	4
7. Imágenes	4
8. Documentos y Videos	4
9. Estructura de Folders Básica	4
A. Estructura del repositorio	7
B. Ejemplo de uso de Git para la entrega de prácticas	7
B.1. Caso de ejemplo	7
B.2. Clonar el repositorio	8
B.3. Crear la rama de la práctica	8
B.4. Realizar cambios y guardar el trabajo	8
C. Subir los cambios a GitLab	8
C.1. Crear Pull Request	8
C.2. Preparar la siguiente práctica	8
D. Branching model diagram	9

Objetivo

Este documento detalla cómo debes participar en los foros, entregar tareas y exámenes, organizar tu código, y subir archivos en la plataforma. Todo el trabajo es incremental y se basa en un solo repositorio en GitLab.

1. Foros de Discusión

Participación semanal obligatoria en cada tema de discusión. Se espera que participes activamente explicando conceptos clave para enriquecer la conversación con tus compañeros.

- Deben responder a cada uno de los puntos de todas las discusiones de cada semana en la plataforma.
- Una respuesta por cada discusión.
- Ejemplo: En la semana 1 tienen 2 discusiones y en cada discusión también deben explicar 3 conceptos o temas para poder tener una mejor discusión.
- Deben seguir las instrucciones específicas de cada semana.

2. Foros de Discusión por Pares

Colabora y genera interacción entre compañeros. Este espacio está destinado a compartir ideas, resolver dudas y reforzar conocimientos a través del trabajo en equipo.

- Deben contestar en el foro de un compañero (puede ser más de uno) y responder cuando se les haga preguntas.

3. Forma de Entrega del Código

Todo el trabajo se concentra en un único repositorio en GitLab. Esto garantiza consistencia, seguimiento del progreso y una evaluación justa.

- Se debe crear un repositorio (solo para el Proyecto Capstone).
- El repositorio debe ser creado en **GitLab**.
- Se debe dar permisos al profesor y al practitioner correspondiente. Los contactos disponibles son:

Role	Email
Professor: Einar Rocha	einar.rocha@jala.university
Practitioner: Carlos Zurita	carlos.zurita@jala.university
Practitioner: Luis Morales	luis.morales@jala.university
Practitioner: Juan Esteves Giratá	Juan.Esteves@jala.university
Practitioner: Alvaro Quispe	Alvaro.Quispe@jala.university

Nota importante: GitLab solo permite agregar a dos colaboradores en proyectos privados. Por lo tanto, debes asignar únicamente al docente y al practitioner correspondiente.

- Si no se otorgan permisos a los contactos indicados, la calificación será de **0 (cero)**, ya que no se podrá visualizar la tarea.
- Dependiendo de la semana, el profesor o los practitioners revisarán la tarea semanal y deben tener acceso.

4. Entrega en la Plataforma

La entrega debe ser un único link al Pull Request correspondiente de la semana. En el contexto de GitLab un Pull Request es un Merge Request.

- En la plataforma se debe entregar solo el link al Pull Request. Ejemplo:

```
https://gitlab.com/juan-peres/capstone/-/merge_requests/1
```

- No enviar links que no sean Pull Request.
- No enviar documentos, PDF, imágenes en la plataforma.
- No crear más repositorios, solo uno.
- Si alguna tarea o examen indica crear un repositorio en GitHub, **ignorar esa instrucción**. Solo se manejará un repositorio para el proyecto en GitLab.

5. Configuración de .gitignore

Protege tu repositorio y evita subir archivos innecesarios.

Ejemplo básico de .gitignore

```
# Node modules
node_modules/

# Archivos de entorno
.env

# Logs
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Cobertura de código
coverage/

# Archivos del sistema operativo
.DS_Store
Thumbs.db

# Resultados de pruebas (si se usan herramientas como Jest o
  ↪ Mocha)
test-results/
```

Notas importantes:

- Nunca deben subirse las carpetas `node_modules` ni `coverage` al repositorio.
- El archivo `.env` puede contener claves o variables sensibles, y debe ser siempre ignorado.
- Si se generan otros archivos o carpetas temporales durante el desarrollo o pruebas, también deben agregarse al `.gitignore`.
- Configurar correctamente el archivo `.gitignore` para excluir dicha carpeta.

6. Exámenes

Los exámenes se basan en el mismo proyecto. Cada entrega agrega nuevas funcionalidades, por lo que debes mantener tu código actualizado y ordenado.

- Los exámenes son incrementales sobre el proyecto (código adicional sobre el mismo código del proyecto).
- Todo el trabajo de esta materia es incremental.

7. Imágenes

- Pueden incluir imágenes en el Pull Request.

8. Documentos y Videos

Guarda videos solicitados en exámenes y colecciones de Postman en una carpeta específica del repositorio.

- Pueden subir videos (se solicitan en los exámenes) y colecciones de Postman en el folder `documents`.

9. Estructura de Folders Básica

Este es la estructura de carpetas que debería de tener el proyecto.

```
capstone/  
|-- documents/  
|   |-- myVideo.mp4  
|   |-- postman.json  
|-- images/  
|   |-- coverage.jpg  
|-- src/  
|   |-- controllers/  
|   |   |-- userController.js  
|   |   |-- authController.js  
|   |-- middlewares/  
|   |   |-- authMiddleware.js  
|   |-- models/  
|   |   |-- userModel.js
```

```
|  |-- routes/
|  |  |-- userRoutes.js
|  |  |-- authRoutes.js
|  |-- services/
|  |  |-- userService.js
|  |-- utils/
|  |  |-- logger.js
|  |-- config/
|  |  |-- dbConfig.js
|  |-- tests/
|  |  |-- myTest.test.js
|  |-- app.js
|  |-- server.js
|-- .env
|-- .gitignore
|-- package.json
|-- package-lock.json
|-- README.md
```

Anexo A

Datos Complementarios para el manejo delCodigo

A. Estructura del repositorio

Cada estudiante deberá crear un repositorio propio en GitLab llamado: **Capstone**

Manejo de ramas

El repositorio debe utilizar ramas para el desarrollo de cada práctica siguiendo estas reglas:

- La rama principal se llamará **master**.
- Por cada práctica se creará una rama a partir de **master**, con el siguiente formato: `<InicialNombre><ApellidoPaterno>-<NombreDelProyecto>-practica-<numero>`.

MPerez-UNO-practica-1

- Cada práctica se construye sobre la anterior, por lo que la rama de la práctica siguiente debe partir de **master** una vez que el Pull Request anterior haya sido fusionado.

Entrega de prácticas

- Al finalizar cada práctica, se deberá crear un Pull Request (PR) desde la rama correspondiente hacia **master**.
- El enlace al PR debe ser publicado en la plataforma en la practica correspondiente antes de la fecha límite establecida para cada práctica.
- El PR sera revisado por el docente para asignar la nota respectiva.
- El PR debe contener una descripcion de los principales cambios que se realizado para cumplir con la practica, puede agregar capturas de pantallas.
- Si las capturas de pantalla corresponden a porciones de codigo, es necesario que diga en que clase o archivo esta ese codigo.

Al crear el PR debe verificar que los Merge Options se muestren como en la imagen:

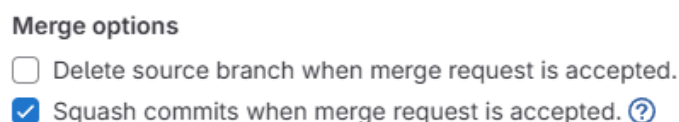


Figura 1: Merge Options.

B. Ejemplo de uso de Git para la entrega de prácticas

B.1. Caso de ejemplo

Supongamos que la estudiante María Rene Pérez Ramos trabaja en el proyecto **UNO** y va a desarrollar la práctica 1.

B.2. Clonar el repositorio

```
git clone https://github.com/MariaPerez/Capstone.git
cd Capstone
```

B.3. Crear la rama de la práctica

```
git checkout -b "mperez-one-practica-1"
```

B.4. Realizar cambios y guardar el trabajo

```
# Editar archivos, agregar funciones, etc.
git add .
git commit -m "Practica 1: implementacion inicial"
```

C. Subir los cambios a GitLab

```
git push origin mperez-one-practica-1
```

C.1. Crear Pull Request

Desde la plataforma de GitLab:

- Ir al repositorio.
- Seleccionar "Compare & pull request".
- Crear un PR desde `mperez-one-practica-1` hacia `master`.
- Agregar un mensaje claro con los cambios realizados.
- Agregar al docente como revisor.
- Verificar los merge-options.
- Publicar el link del PR en la plataforma.
- Puedes fusionar el PR, y empezar a trabajar en la siguiente practica.

C.2. Preparar la siguiente práctica

Una vez aprobado y fusionado el PR:

```
git checkout master
git pull origin master
git checkout -b mperez-one-practica-2
```

D. Branching model diagram

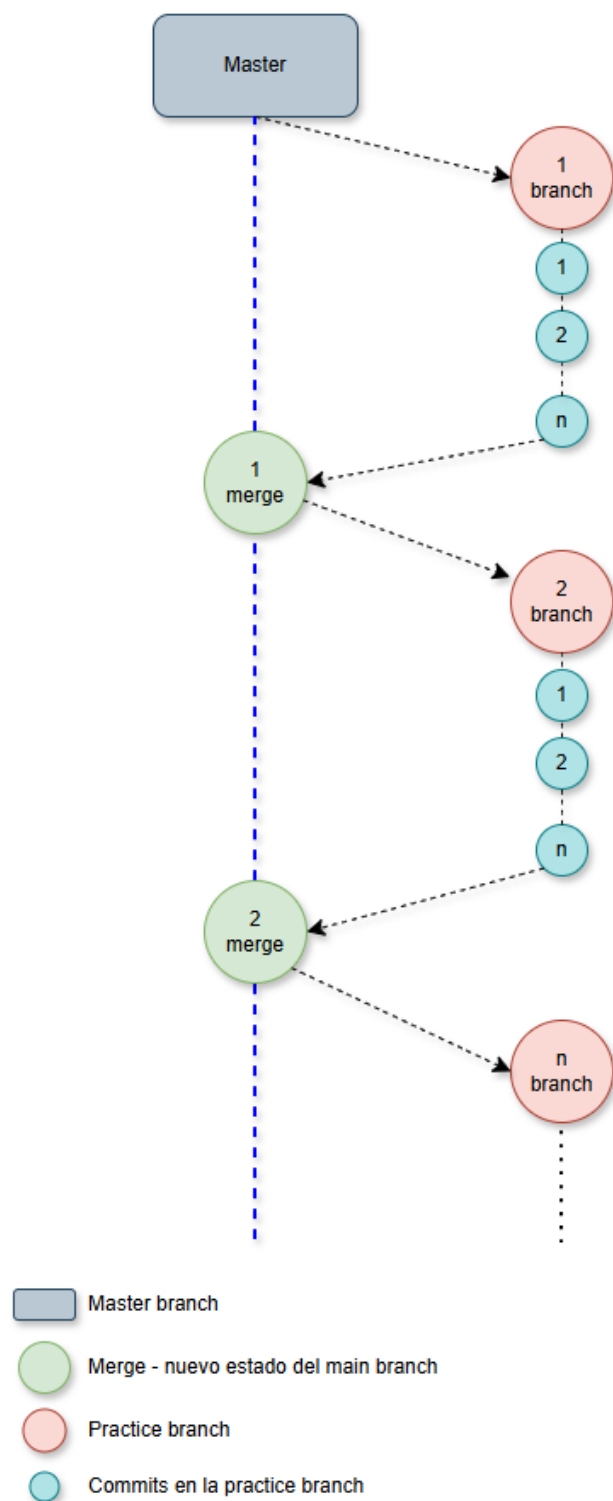


Figura 2: Flujo.